Deep Learning for Automatic Seal Counting in Aerial Images

Sophie Bickerton (170001861)

Supervisors: Dr Kasim Terzíc and Prof Len Thomas

April 2021

University of
St Andrews

Abstract

One of the focuses of the Sea Mammal Research Unit (SMRU) is in monitoring the populations of seals in the U.K.. The aim of this SH project is to research object detection and image classification techniques in order to apply this to the application of aerial images. In order to count the different types of seals, two stage of machine learning are connected together in order to perform the classification successfully. The first stage is using YOLOv3 to detect seals against the background of large aerial images. The detected seals are then cut out of the large aerial images into smaller tiles. These tiles are converted to vectors using a histogram of oriented gradients. The results of this are put through a second stage classification to detect what type of seal it is.

Declaration

I declare that the material submitted for assessment is my own work except where credit is explicitly given to others by citation or acknowledgement. This work was performed during the current academic year except where otherwise stated. The main text of this project report is 11,169 words long. In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web. I retain the copyright in this work.

Acknowledgements

I would like to thank my supervisors, Prof. Len Thomas and Dr. Kasim Terzíc for their continued support and advice throughout this turbulent year. I appreciate their guidance and insights in this field and the results of this project were greatly impacted by our discussions throughout the course of the year.

Contents

1 Introduction **7**
1.1 Disruption Due to the COVID-19 Pandemic . . . 7

2 Context Survey **8**
2.1 Standard Machine Learning Algorithms . . . 8
2.1.1 Support Vector Machine . . . 8
2.1.2 Random Forest . . . 9
2.2 Deep Learning Overview . . . 10
2.2.1 Neural Network Architecture . . . 12
2.2.2 Universality Theorem . . . 13
2.2.3 Forward and Back Propagation . . . 13
2.3 Convolutional Neural Networks . . . 17
2.3.1 Introduction . . . 17
2.3.2 Architecture . . . 17
2.4 Object Detection . . . 19
2.4.1 Overview . . . 19
2.4.2 Performance Metrics . . . 21
2.4.3 YOLO . . . 23
2.5 Summary of Prior Work . . . 25

3 Requirements Specification **26**
3.1 Primary Objectives . . . 26
3.2 Secondary Objectives . . . 26
3.3 Primary Requirements . . . 26
3.4 Secondary Requirements . . . 27
3.5 Tertiary Requirements . . . 27

4 Software Engineering Process **28**
4.1 Development Process . . . 28
4.2 Resources and Technologies . . . 28

5 Ethics **29**

6 Design **30**
6.1 Data Preparation . . . 30
6.1.1 Image Cropping . . . 30
6.1.2 Tile Batches . . . 30
6.1.3 Anchors . . . 30
6.2 Object Detection . . . 30
6.2.1 YOLO Architecture . . . 31
6.2.2 Training . . . 31
6.2.3 Evaluation . . . 31
6.3 Transformation Stage . . . 32
6.3.1 Prediction Cropping . . . 32
6.3.2 Extracting HOG . . . 32
6.4 Image Classification . . . 32
6.4.1 Support Vector Machine . . . 32

6.4.2 Random Forest . . . 33
6.4.3 Neural Networks . . . 34

7 Implementation **35**
7.1 Data Preparation . . . 35
7.1.1 Image Cropping . . . 35
7.1.2 Tile Batches . . . 35
7.1.3 Anchors . . . 35
7.2 Object Detection . . . 36
7.2.1 Training . . . 36
7.2.2 Evaluation . . . 36
7.3 Transformation Stage . . . 36
7.3.1 Prediction Cropping . . . 36
7.3.2 Extracting HOG . . . 37
7.4 Image Classification . . . 37
7.4.1 Support Vector Machine . . . 37
7.4.2 Random Forest . . . 38
7.4.3 Neural Networks . . . 38

8 Evaluation and Critical Appraisal **39**

1 Introduction

The Seal Mammal Research Unit (SMRU) uses aerial images to count seals and classify their stage of development (pups, whitecoats, moulted, etc.) in order to ultimately estimate the population of seal in Scotland. Currently, around 40% of all grey seals live in UK waters so estimating this population is significant.[Scot, 2020] This task is mainly completed by hand and takes a significant amount of time to complete each year. To replace this process, the implementation will be to detect and classify seals from the provided aerial images. Steps towards this goal have been made previously by two students from St. Andrews, Dorottya Denes and Samuel Pavlik, whose work has been built upon in this project.[Denes, 2020, Pavlik, 2019] This paper also looks in depth at a variety of neural network and machine learning algorithms to analyse which are appropriate for the application.

1.1 Disruption Due to the COVID-19 Pandemic

This entirety of this project was conducted during the pandemic and as a consequence planning was done with the consideration that all work would have to be completed remotely. Therefore, there was no disruption to the project in terms of not being able to carry what was initially planned.

2 Context Survey

Machine learning is a branch of Artificial Intelligence (AI) which applies statistics to algorithms in order to find patterns within data. This project uses machine learning techniques in order to detect and classify seals. Due to this, the context survey looks into standard machine learning algorithms, deep learning and specialist object detection algorithms.

2.1 Standard Machine Learning Algorithms

In this project the focus is mainly on supervised learning. This is when the output being predicted is known and therefore algorithms can be trained using both the inputs and outputs. Supervised learning algorithms can be typically split into two types: regression and classification. In both cases the algorithm takes an input in order to produce a predicted output, which can then be compared to the true value. In the case of regression, the output is a real value, belonging to a continuous range, the range of which is dependent on the context. In the case of classification, the output is a label which can be used to group inputs (e.g., 'car', 'train', 'bicycle'). The following sections highlight key algorithms which can be used to classify inputs.

2.1.1 Support Vector Machine

The concept of the Support Vector Machine (SVM) is to find a hyperplane which divides the different classes. Sectioning the input space in this manner enables the algorithm to predict which class an input belongs to. With 2 input features (e.g., height and width) the hyperplane is a line which separates the classes, the more features mean the hyperplane is in more dimensions. Taking the simply 2-D case, SVM works to find the 'best' hyperplane, though this is more difficult than it sounds. Between the two classes there will be a gap where several lines of several orientations can be drawn. Figure 1 demonstrates this problem.

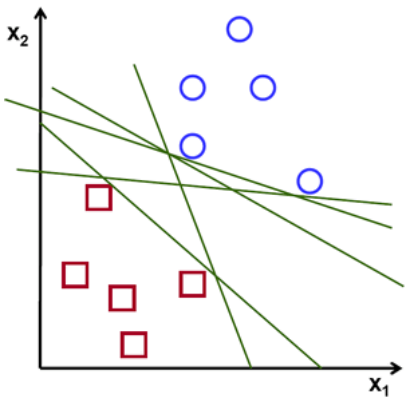


Figure 1: 2 feature hyperplanes Source:[Gandhi, 2018]

SVM uses the points closes to the hyperplanes, called support vectors, in order to calculate the optimal hyperplane. This is done by drawing planes through the support vectors of each class which are parallel. The optimal hyperplane is then equidistant to the two support vector lines. The result is shown in Figure 2.

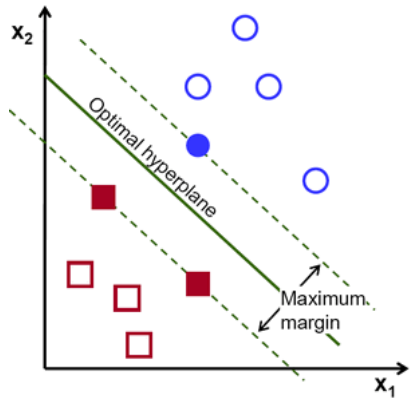


Figure 2: Optimal Hyperplane Source:[Gandhi, 2018]

In the SVM algorithm the aim is to maximise the margin between the support vectors. Therefore, the cost/error/loss function and any updates to the gradient of the hyperplane are based on this. The loss function for the SVM is called hinge loss and is given by equation 1. This illustrates that if the predicted value, f(x), is the same sign as the actual value, y then the loss is 0.

$$L(x, y, f(x)) = \begin{cases} 0 & \text{if } y * f(x) \geq 1 \\ 1 - y * f(x) & \text{otherwise} \end{cases} \tag{1}$$

2.1.2 Random Forest

The concept of the random forest algorithm is that there are several decision trees which are produced from random samples of the dataset. The sampling technique is bootstrapping where there may be repeated entries in the training sample, this is also known as sampling with replacement. Each decision tree is trained on these samples and with an input predicts a class as an output. This is called a 'vote', a majority of the votes decides the class that is predicted by the algorithm.[Caie et al., 2021] When the decision trees are trained, they are done so without pruning, this means that the depth and overall size of the tree is not restricted.

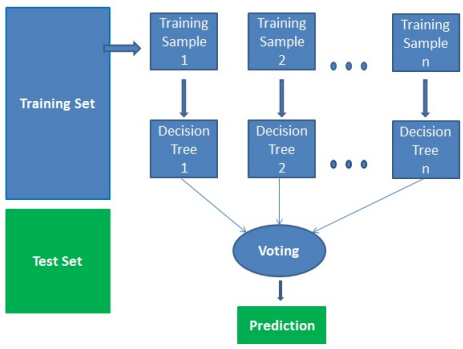


Figure 3: Architecture of the Random Forest algorithm Source:[Navlani, 2018]

2.2 Deep Learning Overview

Deep learning is a subsection of machine learning which focuses on algorithms and techniques that focus on artificial neural networks. These techniques attempt to imitate the way the human brain processes data, spots patterns and trends and can put this together to identify or classify object. The study of how the human brain works is a substantial part of neuroscience. There has been research into how the brain is made up of nerve cells, called neurons which was initially discovered back in 1873.[Golgi, 1873] A little later the nerve cell model was formed in the "neuronal doctrine" which demonstrated the structure of each nerve cell that makes up the nervous system.[y Cajal, 1888]

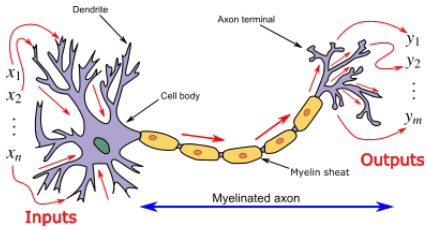


Figure 4: Nerve Cell Model[Commons, 2018]

Figure 4 demonstrates that the human nervous system is not continuous, as was previously thought, but made up of billions of nerve cells which are connected to each other. These series of cells can lead to human having thoughts, making decisions and completing tasks. Taking this as a base, the link between nerve cells, or neurons, and neural networks can be seen. The neural network model was first proposed by McCulloch and Pitts in 1943.[McCulloch and Pitts, 1943] This model proposed an artificial neuron which could be used in a network in order to replicate the complex decisions and tasks humans could make.

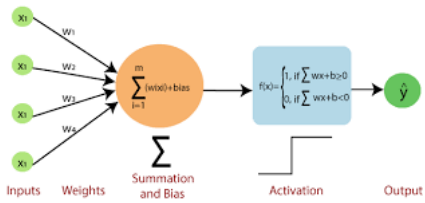


Figure 5: Model of a Perceptron Source:[Point, 2021]

The model in Figure 5 is described as a perceptron, made up of an input function, activation function and relevant weighting.[Noriega, 2005]

Input Layer

Each of the inputs in the input layer have weighting applied to them at the input function.

$$z = \sum_{i=1}^{n} w_i * x_i + b$$

The input function is the summation of the weighted inputs plus a bias weight, b. Changing the value of the bias weight shifts the threshold of a and can completely change the output of the perceptron. The result of this function, which is usually a summation of the weighted inputs, is then passed to the activation function.

Hidden Layer

The activation function as presented by McCulloch and Pitts was a simple threshold function. [McCulloch and Pitts, 1943]

$$threshold\,function = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

This threshold function determines if the result should be 1 or 0 based on the result of the summation plus a bias weight. In the example above the threshold is -b, as the summation only has to be larger than -b in order for the results to be greater than 0. However, it is common for the activation function one of a variety of options. Some of the popular options are the logistic(sigmoid), hyperbolic tangent (tanh), rectified linear activation function(ReLU) and softmax.[Russell and Norvig, 2002]

$$sigmoid = \frac{1}{1 + \exp(-z)}$$

The advantage of the sigmoid function is that it is non-linear so produces a range of values from 0 to 1. By changing any value of a to be between 0 and 1 normalizes it, but this can result in extreme values not being recognised and so error with the resulting prediction. This is called the vanishing gradient problem (see 2.2.3).

$$tanh = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

Similar to the sigmoid function, the hyperbolic tangent is a non-linear function. However, it instead ranges from -1 to 1 which means this function is less susceptible to having a vanishing gradient.

$$ReLU = max(0, z)$$

Rectified linear activation function produces 0 for any value that is less than or equal to 0 and the value of the input function otherwise. Because of the nature of producing linear values when $a > 0$, and 0 otherwise, ReLU is categorised as a piecewise linear function. The reason this function is preferred is that it is much less susceptible to the vanishing gradient problem than sigmoid and tanh functions. ReLU is also much less computationally intensive due to only requiring the max function. It can therefore be a much better option for training neural networks with several hidden layers. However, when using back-propagation it can cause the dying ReLU problem (see 2.2.3). Therefore, a workaround can be used in the form of Leaky ReLU.

$$Leaky\,ReLU = max(0.1z, z)$$

Instead of 0 being output for values less than or equal to 0, it is one tenth of the initial value. This means the tail allows for negative values and prevents the dying ReLU problem.

$$Softmax = \frac{\exp(z_i)}{\sum_{j=1}^{k} \exp(z_j)}$$

Unlike the previous functions, softmax is an activation function that best suits multi-class classification. It takes in several values but also outputs several values. Softmax uses all the inputs to determine the weighting that each one should have. Using the formula, it weights each of the inputs and calculates a probability for each one. Therefore, the larger the input value, the greater the probability. It is worth noting that all values are between 0 and 1 and the sum of all probabilities is 1. Due to the result of several values the softmax function is often used in the output layer to determine which class a given input is in.

The results of the activation function can then be passed on as an output, $\hat{y}$ for that perceptron. Due to all of the different options for the perceptron, it is fairly intuitive to see that combine many perceptrons together can result in a range of complex functions. This can result in several hidden layers of perceptrons making up a neural network.

2.2.1 Neural Network Architecture

A neural network can be formed in several ways using the perceptron as the basic unit. We will discuss the different elements that make up the network.

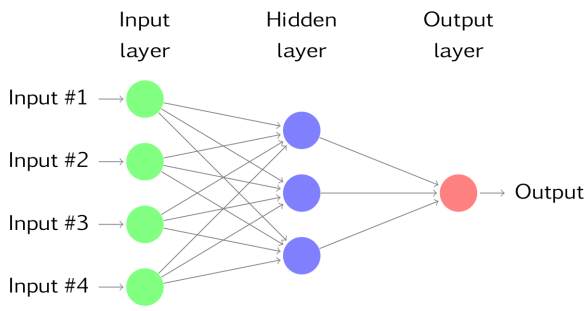


Figure 6: Model of a Neural Network Source: [Hyndman and Athanasopoulos, 2018]

Figure 6 is a diagram which provides an overview of what a neural network will look like. This is a simple version but can be further expanded and is an example of a feed-forward network. There are 3 key layers here, the input, hidden and output layers. The input layer consists of several individual inputs x_i, these then each have weights applied to them. The hidden layer is made up of several perceptrons which each convert the inputs to one or more outputs. In Figure 6 there is one hidden layer made up of 3 hidden neurons (perceptrons). There can be several hidden layers and a variable number of hidden neurons in each. As can be seen above, each input feeds into each of the hidden neurons which each produce a single output to feed into the output layer. In a case where there are several hidden layers, each of the hidden neurons feed the output into each of the neurons in the next layers. For example, if there are 4 hidden neurons in the second layer then each neuron in the first hidden layer feeds $\hat{y}_i$ 4 times into the 4 hidden neurons. Finally, the output layer takes the outputs of the last hidden layer and made up of perceptrons. Therefore, it weights the values from the hidden layer in order to produce one final value (or class in the case of classification, but more on that later).

2.2.2 Universality Theorem

We have now looked into how artificial neural networks work but it is important to discuss what the advantages are. After the concept of the perceptron was presented by McCulloch and Pitts, it was converted into an algorithm and put into place in 1958 by Frank Rosenblatt, in the form of the Mark 1 Perceptron.[Rosenblatt, 1958] However, this application was limited as the results could only separate classes in a linear manner. One major fallback was that this application could not represent XOR (exclusive-or) and this led to a thaw in development until the 1980s.[Kotu and Deshpande, 2019] This is when the concept of multi-layer perceptrons came into fruition. This is a class of feed-forward neural networks where there is more than one perceptron in the network and in different layers. Figure 6 is an example as there are perceptrons in the hidden and output layers. Having several layers of perceptrons make the algorithm more adjustable and with a variety of activation functions, can be used to represent XOR, as well as many more tasks. The universality theorem summarises this well, "no matter what function we want to compute, we know that there is a neural network which can do the job".[Nielsen, 2015] The neural network, however, will only be able to approximate the function and the structure of this network will determine how good the approximation is. For example, a network with 5 hidden neurons will make a better approximation that a network with 3 hidden neurons. Generally, a function, f, is approximated well by the network if $|g(x) - f(x)| < \epsilon$, where g is the approximated function and ϵ is the desired accuracy.[Nielsen, 2015]

What to do next: Then explain back/forward propagation with gradient descent and loss.

2.2.3 Forward and Back Propagation

Forward-Propagation

The structure of the neural network has been explained, with an example of a feed-forward network. To understand forward-propagation we will utilise a similar example from Figure 7.

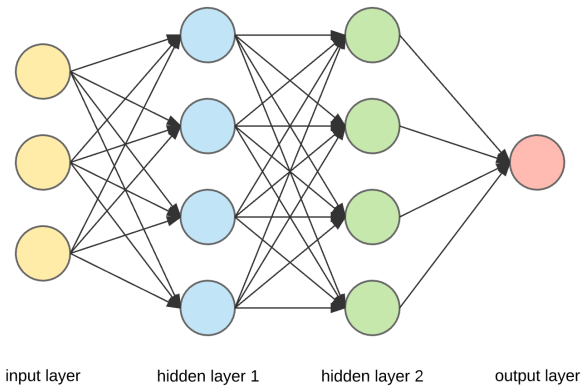


Figure 7: Neural Network with 2 hidden layers, each with 4 hidden neurons Source:[Eliassy, 2019]

Firstly, let the inputs be defined as $\boldsymbol{X} = [x_1, x_2, x_3]^T$, the output from the first hidden layer be defined as $\boldsymbol{A^{(1)}} = [a_1^{(1)}, a_2^{(1)}, a_3^{(1)}, a_4^{(1)}]^T$ and the output from the second hidden layer be defined as $\boldsymbol{A^{(2)}} = [a_1^{(2)}, a_2^{(2)}, a_3^{(2)}, a_4^{(2)}]^T$. Each input has a different weighting, so from the input layer to the first hidden layer let the weights be defined as follows:

$$\boldsymbol{W} = \begin{bmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} & w_{1,3}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} & w_{2,3}^{(1)} \\ w_{3,1}^{(1)} & w_{3,2}^{(1)} & w_{3,3}^{(1)} \\ w_{4,1}^{(1)} & w_{4,2}^{(1)} & w_{4,3}^{(1)} \end{bmatrix}$$

where $w_{i,j}^{(k)}$ is the weight of input j at hidden neuron i in hidden layer k. Therefore, the formula for each hidden neuron, $a_i^{(1)}$ is:

$$\begin{aligned} a_1^{(1)} &= f(w_{1,1}^{(1)}x_1 + w_{1,2}^{(1)}x_2 + w_{1,3}^{(1)}x_3 + b_0) \\ a_2^{(1)} &= f(w_{2,1}^{(1)}x_1 + w_{2,2}^{(1)}x_2 + w_{2,3}^{(1)}x_3 + b_0) \\ a_3^{(1)} &= f(w_{3,1}^{(1)}x_1 + w_{3,2}^{(1)}x_2 + w_{3,3}^{(1)}x_3 + b_0) \\ a_4^{(1)} &= f(w_{4,1}^{(1)}x_1 + w_{4,2}^{(1)}x_2 + w_{4,3}^{(1)}x_3 + b_0) \end{aligned}$$

where f is the activation function and b_0 is the bias for that layer.

From the first layer to the second the weighting that is applied is as follows:

$$\boldsymbol{W} = \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} & w_{1,3}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} & w_{2,3}^{(2)} \\ w_{3,1}^{(2)} & w_{3,2}^{(2)} & w_{3,3}^{(2)} \\ w_{4,1}^{(2)} & w_{4,2}^{(2)} & w_{4,3}^{(2)} \end{bmatrix}$$

Therefore, the formula for each hidden neuron, $a_i^{(1)}$ is:

$$\begin{aligned} a_1^{(2)} &= f(w_{1,1}^{(2)}x_1 + w_{1,2}^{(2)}x_2 + w_{1,3}^{(2)}x_3 + b_1) \\ a_2^{(2)} &= f(w_{2,1}^{(2)}x_1 + w_{2,2}^{(2)}x_2 + w_{2,3}^{(2)}x_3 + b_1) \\ a_3^{(2)} &= f(w_{3,1}^{(2)}x_1 + w_{3,2}^{(2)}x_2 + w_{3,3}^{(2)}x_3 + b_1) \\ a_4^{(2)} &= f(w_{4,1}^{(2)}x_1 + w_{4,2}^{(2)}x_2 + w_{4,3}^{(2)}x_3 + b_1) \end{aligned}$$

The output layer is then:

$$\hat{y} = g(\theta_1 a_1^{(2)} + \theta_2 a_2^{(2)} + \theta_3 a_3^{(2)} + \theta_4 a_4^{(2)} + b_2)$$

where θ_i is the weighting for a_i.

In this example we can see how the results of each layer are fed forward to the next layer, culminating with the final output. This is forward-propagation. However, when addressing the potential error of $\hat{y}$ we need to use back-propagation in order to improve the accuracy.

Back-propagation was introduced in machine learning back in 1986 in order for neural networks to be able to learn internally and by adjusting weights and biases.[Rumelhart et al., 1986] Using back-propagation requires an error (also known as cost) function. This function will give a value to represent the error between the provided output and the expected output. This is typically the mean squared error for linear regression:

$$C(X, \theta) = \frac{1}{N} \sum_{i=1}^{N} (y - \hat{y})^2$$

where N is the number of outputs. Or for binary classification tasks it is typically cross-entropy:

$$C(X,\theta) = \frac{1}{m}\sum_{i=1}^{m}[y \ln a_i^{(j)} - (1-y)\ln 1 - a_i^{(j)}]$$

where m is the number of classes, y the expected output and a_i^j is the output of node i in layer j (so $\hat{y}$ in the output layer).[Nielsen, 2015]

The aim of back-propagation is to minimize this cost function, using what is called the optimization function and this is typically done through a method called gradient descent.

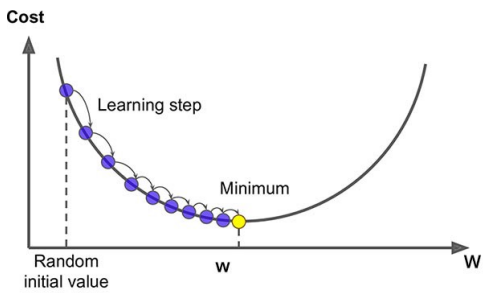


Figure 8: Diagram of Gradient Descent Source:[Sauer, 2020]

Gradient descent works from a given initial value and steps towards the minimum. At each step the function calculates the gradient and moves in the negative direction. The step size is predetermined by a value called the learning rate. The greater the learning rate the larger the steps that are taken. The optimization function iterates such that it moves towards the minimum until the difference in the cost function of the current step and previous step is below a given threshold. This value is then identified as the minimum. Back-propagation applies this optimization function from the output layer and back through the hidden layers to find the optimum weights at each stage.

Whilst this method is fairly simple at the output layer, when going back through the hidden layers it can become more complex. For each subsequent layer, the derivatives of the layers to the right of it are also applied since the weights are connected. Calculating the gradient $\frac{\partial C(X,\theta)}{\partial w_{i,j}^{(k)}}$ is no longer as simple as it may seem.

We derive the gradient by applying the chain rule.

$$\frac{\partial C(X,\theta)}{\partial w_{i,j}^{(k)}} = \frac{\partial C(X,\theta)}{\partial s_i^{(k)}}\frac{\partial s_i^{(k)}}{\partial w_{i,j}^{(k)}}$$

where $w_{i,j}^{(k)}$ is the weight j for node i in layer k and $s_i^{(k)}$ is the results of the product-sum plus bias

for node i in layer k.

$$
\begin{aligned}
Let\, \delta_i^{(k)} &= \frac{\partial C(X,\theta)}{\partial s_i^{(k)}} \\
\frac{\partial s_i^{(k)}}{\partial w_{i,j}^{(k)}} &= \frac{\partial}{\partial w_{i,j}^{(k)}} \Big[\sum_{l=0}^{n} w_{l,j}^{(k)} a_l^{(k-1)} \Big] = a_i^{(k-1)} \\
\frac{\partial C(X,\theta)}{\partial w_{i,j}^{(k)}} &= \delta_i^{(k)} a_i^{(k-1)}
\end{aligned}
$$

where n is the number of nodes in the previous layer. As can be seen from this series of equations, for each layer that is back-propagated, more derivatives are required in order to find the minimum for that weight.

Vanishing Gradient Problem

When using an activation function such as sigmoid in multiple nodes in multiple layers, it can cause problems in back-propagation. Sigmoid is a function which converts the result of the input function between 1 and 0. Therefore, changing the weights will only see a small change to the resulting output due to this small output space.

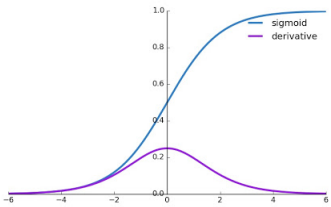


Figure 9: Diagram of Sigmoid function and its' derivative Source:[Learning*, 2018]

The vanishing gradient problem occurs as the derivative of the sigmoid function is very small. If there are several layers that use the sigmoid as the activation function, then it results in the multiplication of increasingly small values. Therefore, the initial weights will not be minimized correctly and as such the model will be inaccurate.

Dying ReLU Problem

This problem is very similar to the vanishing gradient problem. The ReLU activation function produces 0 as the result for input values that are less than or equal to 0. This also subsequently means that the derivative is also 0 for these values. Therefore, when calculating the loss, as the derivative is 0 the loss will mistakenly be calculated as 0 for that node and all nodes to the left of it during back-propagation. Therefore, the weights will not be altered, and this results in the model not being fully optimized.

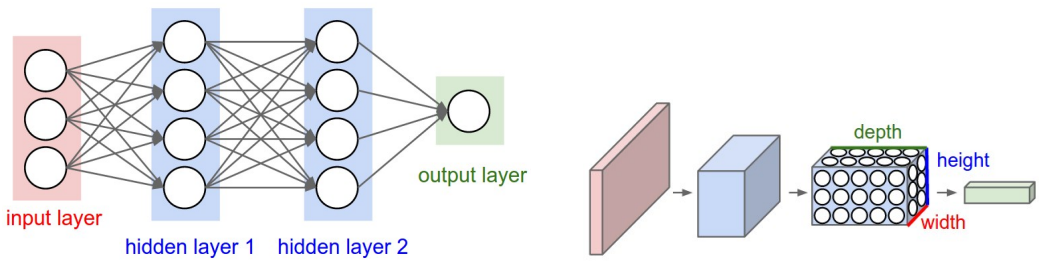
2.3 Convolutional Neural Networks

2.3.1 Introduction

Convolutional Neural Networks (CNNs) are a subsection of Artificial Neural Networks that are ideal for computer vision-based tasks.[Albawi et al., 2017] The name convolutional comes from the use of the mathematical process involving matrices called convolution. The CNNs are similar to ANNs in that they are made up of neurons, each of which has inputs, learnable weights and biases and an output. The difference with CNNs is that is it designed to take an image as the initial input and as such the rest of the layers are designed to handle this type of input.

2.3.2 Architecture

As mentioned in the introduction, the architecture of CNNs is designed in order to be able handle images as input and detect objects/patterns accordingly.



(a) Neural Net with 2 hidden layers, each with 4 nodes

(b) Convolutional Neural Network

Figure 10: Comparison of Neural Networks Source:[University, 2021]

CNNs are made up of three key layers: a Convolutional layer, a Pooling layer and a Fully-Connected layer. However, unlike ANNs there can be multiple instances of each layer and they can be interspersed throughout the architecture. An example of a CNN is Convolutional, Pooling, Convolutional, Pooling, Fully-Connected and Fully-Connected.

Convolutional Layer

An image can be thought of as a series of layers. An image is made up of pixels and these pixels have values corresponding to colour channels. These colour channels can be RGB, Greyscale, CYMB etc. In the case of RGB an image has a red layer, a green layer and a blue layer and values for each pixel in these layers. The concept of the convolutional layer is to pass a filter (also known as a kernel) over each of the layers to produce an altered set of pixels as an output.

Image Matrix

1	0	0	1	1
1	0	1	1	1
0	1	1	0	1
0	1	0	1	0
1	1	1	0	1

Filter Matrix

1	0	0
1	0	0
0	1	1

Figure 11: Image and filter matrices example Source:[Walters, 2019]

1	0	0	1	1
1	0	1	1	1
0	1	1	0	1
0	1	0	1	0
1	1	1	0	1

(a) Step 1

1	0	0	1	1
1	0	1	1	1
0	1	1	0	1
0	1	0	1	0
1	1	1	0	1

(b) Step 2

Figure 12: Convolution of image matrix Source:[Walters, 2019]

To look at an example, Figure 11 has an image matrix on the left and a filter matrix on the right. The filter is applied to the top left 3x3 pixels as in Figure 12. The new altered pixel is the product-sum of each of the image (input) pixels and each of the filter pixels. Therefore, the first pixel is: $1 \times 1 + 0 \times 0 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 0 + 0 \times 0 + 1 \times 1 + 1 \times 1 = 4$. The filter matrix will then shift to the next block of 3x3 pixels, and the calculation is completed again. The filter matrix passes over the first row and then shifts down by one pixel to continue the calculation. For the example we have, the resulting altered matrix is:

$$\begin{bmatrix} 4 & 1 & 2 \\ 2 & 2 & 3 \\ 2 & 3 & 2 \end{bmatrix}$$

In the example above, the filter moved across by 1 pixel each time, this movement is called the stride length and can be adjusted. In the case of an image with multiple channels (RGB), there can be a different filter for each channel, but the same process happens. The results for each channel are the added together for that filter position in order to give the altered result. It is also common for several filters to be applied to each channel of image at a single convolutional layer.

Having several convolutional layers in the architecture means that more complex patterns can be detected, such as specific animals and shapes. The convolutional layers at the start tend to detect low level patterns such as edge detection but passing this on to further layers enables the complex pattern detection.

Pooling Layer

Using multiple filters in the convolutional layer can mean the output has a large depth (the depth is typically the number of filters applied) and this can end in a large amount of data that needs to be processed. Pooling the data will substantially reduce what needs to be processed at the next layer. The pooling layer and convolutional layer will typically sandwich a non-linearity (e.g., ReLU), which applies the function to each pixel. If the function is ReLU ($max(0, x)$)this replaces all negative values with 0.

The pooling layer is effectively another filter that passes over the input matrix and again applies a function to the values in the filter to produce an output value. However, the pooling layer typically selects a 2x2 pixel area and has a stride length of 2, so there are no overlaps. There are two types of pooling layer: max pooling and average pooling. These different types perform different functions to the matrix.

Figure 13 shows a matrix which is split into 2x2 sections. Average pooling takes the average of

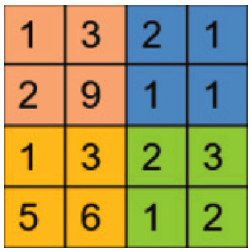


Figure 13: Pooling being applied to a matrix Source:[Li and Zhou, 2019]

each section to form the result. In this example the result would be:

$$\begin{bmatrix} 3.8 & 4.3 \\ 3.8 & 2.0 \end{bmatrix}$$

Max pooling takes the maximum value of each section to form the result. In this example the result would be:

$$\begin{bmatrix} 9 & 2 \\ 6 & 3 \end{bmatrix}$$

Max pooling is often preferred as it is a better noise suppressant, due to ignoring any value that is not the maximum. The result of the pooling layer is passed onto the next layer, such as another convolutional layer or a fully-connected layer.

Fully-Connected Layer

The fully-connected layer is most simply described as a feed-forward neural network. This layer sits at the end of the architecture and is used to produce the predicted output. The output from the final pooling or convolutional layer is a 3-D matrix which is flattened into a column vector.

$$\begin{bmatrix} 9 & 2 \\ 6 & 3 \end{bmatrix} = \begin{bmatrix} 9 \\ 2 \\ 6 \\ 3 \end{bmatrix}$$

This column vector is then the list of inputs (X) for a hidden layer. Each fully-connected layer effectively represents a hidden layer in an ANN. Finally, after passing through the fully-connected layers the final layer in the architecture applies the softmax activation function. This calculates the probability that the image belongs in each of the classes, the largest probability is the one which is predicted.

2.4 Object Detection

2.4.1 Overview

Object detection is one of the main applications of Deep Learning, although not all methods of object detection necessarily require deep learning. Object detection consists of identifying the location of the object, with a bounding box, and then providing a label to said object. This can be simply

be described as a localization step followed by a classification step.

Initially, using features of an image in order to produce numerical output then allows said output to be applied to a machine learning algorithm. Using Gradient Vectors, Histogram of Oriented Gradients (HOG). The gradient vector is calculated for each pixel in an image to give a matrix of gradients. This is a metric for how much the pixel colour changes in the x and y direction. Figure 14 gives an example of how this calculation is done.

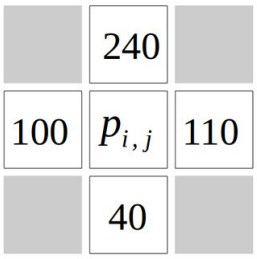


Figure 14: Gradient vector of the pixel $p_{i,j}$ Source:[Ahmadzadeh et al., 2017]

For the x direction the vector is $(110 - 100)$ and for the y direction it is $240 - 40$, the choice of which side to minus from which can be changed as long as it is consistent for the whole image. In this case the resulting gradient vector is $\begin{bmatrix} 10 \\ 200 \end{bmatrix}$ The magnitude and angle can also be calculated:

$$\text{Magnitude} = \sqrt{10^2 + 200^2} = 200.25 \tag{2}$$

$$\text{Angle} = \arctan\left(\frac{10}{200}\right) = 2.86 \text{ degrees} \tag{3}$$

This enable the arrow to be drawn to represent the change in gradient. Gradient vectors can be usefully applied as an edge detector.

HOG converts an initial image into a feature vector of length n, as dictated by the user. In order to calculate the feature vector, the magnitude and angle are calculated for the gradient vectors as seen in equations 2 and 3. For ease, the pixels are typically split into 8x8 pixel squares. Then two 8x8 matrices are constructed as the values of the gradient magnitude and the gradient direction.[McCormick, 2013]

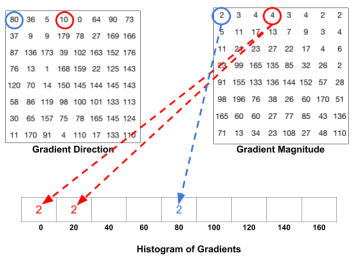


Figure 15: Example gradient magnitude and gradient direction matrices Source:[Weng, 2017]

A row vector is then created to summarize the gradient matrices. According to the value of the gradient direction (angle), the value in the gradient magnitude is added to that bin. If the angle is between two bin values, then the magnitude is divided between the two bins. Going through each pixel in the 8x8 matrices will produce a 9-bin histogram. The final step is called block normalization to make sure that the results are not skewed due to different lighting in the image. Normalizing the vector, $\begin{bmatrix} 128 \\ 65 \\ 234 \end{bmatrix}$, is shown in equations 4 and 5.

$$\sqrt{(128)^2 + (65)^2 + (234)^2} = 274.53 \tag{4}$$

$$\begin{bmatrix} 128 \\ 65 \\ 234 \end{bmatrix} /274.53 = \begin{bmatrix} 0.47 \\ 0.24 \\ 0.85 \end{bmatrix} \tag{5}$$

A 16x16 block of pixels forms 4, 9-bin histograms using the process outlined above. These bins can be concatenated and then normalized using the same technique as normalizing a vector. The block is then moved across by 8 pixels, same as having a stride length of 8, in order to calculate values for the entire image. The final vector of features can then be used with an algorithm such as a SVM in order to classify the image.[Weng, 2017]

Another option is to use a CNN as the architecture makes it a good starting point for object detection. However, the basic CNN has been developed to form subclass which are an improvement on the CNN architecture. One subclass is called R-CNN (Region based CNN) which first detects regions of interest before applying a CNN to classify it, using that region as the input image. R-CNN is designed to reduce the number of proposed regions.[Girshick et al., 2014] This is done using the following algorithm:

```
Selective Search:
1. Generate initial sub-segmentation, we generate many candidate regions
2. Use greedy algorithm to recursively combine similar regions into larger ones
3. Use the generated regions to produce the final candidate region proposals
```

[Girshick et al., 2014]

The regions are identified as groupings of pixels which have similar features (colour, lighting, texture etc.). The final regions are warped into a square before being used as input to the CNN, this is due to the CNN having a standard image size that it works with. The CNN produces a feature vector as an output. This output is then passed to a SVM in order to classify the object. The algorithm also alters the proposed region to be a bounding box for the specific object detected. To improve the proposed bounding boxes, the R-CNN can run linear regression.

2.4.2 Performance Metrics

Performance metrics are a useful tool in order to evaluate a model and find the balance between locating an object and classifying it.

Intersection over Union

When an object is detected, there is a bounding box supplied with it which signifies the location of the detected object. During training and testing, in order to determine whether or not a proposed

bounding box is correct a metric called Intersection over Union (IoU) is applied. This metric compares the position of the proposed bounding box with the position of the true bounding box.

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}} \tag{6}$$

This is the area of overlap divided by the total area the two boxes cover (not counting the intersection twice). A threshold is typically given for the IoU around 0.5 to 0.7, a higher threshold means the bounding boxes have to be more overlapped in order to be set as detected. This tends to increase precision but decrease recall. See 2.4.2.

Confusion Matrix

A confusion matrix is a tool which shows how well a known set has been classified. In the case of evaluating a dataset, the model produces bounding box and class labels which can be compared with the true bounding boxes and labels. In figure 16 it shows how a confusion matrix is laid out with the terminology for each box. This example is when only one class is being detected. For more classes the matrix adds rows.

		Actual	
		Positive	Negative
Predicted	Positive	**True Positive**	**False Positive**
	Negative	**False Negative**	**True Negative**

Figure 16: Diagram of a confusion matrix Source:[Liu, 2018]

Precision and Recall

Precision and recall are two standard metrics which summarise the performance of a model.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{7}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{8}$$

Precision is the number of correctly identified object divided by all identified objects. Having a high precision means that an object is only determined if the model determines it has a high probability. However, this could mean a significant proportion of objects are missed, especially is recall is low. Recall is the number of objects identified divided by all objects in the dataset. Having a high recall means that barely any objects are missed. Having a high precision and recall is the aim of the final output, though in some cases one may be preferred over the other.

A model can be evaluated by these two metrics using a Precision-Recall Curve, these is a plot which shows how precision and recall change with each other. Figure 17 shows an example of this curve. The curve can be used to demonstrate the trade-off between precision and recall and help decide which values the model should take.

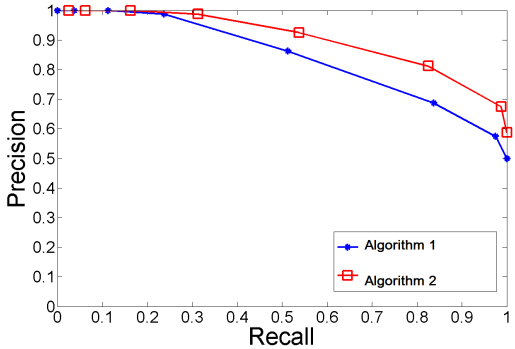


Figure 17: Example of a Precision-Recall Curve Source:[Mr.A and lnathan, 2017]

AP and mAP

Precision and recall are useful metrics but sometimes the Precision-Recall curve can make finding the best values confusing. AP (Average Precision) is a metric which summarises the recall and precision to identify the best combination.

$$\text{AP} = \sum_{i=0}^{n-1} [\text{Recall}(i) - \text{Recall}(i+1)] \cdot \text{Precision}(i) \tag{9}$$

In equation 9 AP is calculated as a loop that goes through all the precision/recall thresholds (e.g., 0.5, 0.6, 0.7, 0.8, 0.9, 1.0) finding the difference between the recall at the current threshold and the next, multiplied by the precision. It is the weighted sum of the precisions where said weight is the difference in recall.

The mean average precision (mAP) is calculated as the mean of the APs of each class in the object detection. The general equation is seen in 10, where N is the number of classes.

$$\text{mAP} = \frac{1}{N} \sum_{k=1}^{N} \text{AP}(k) \tag{10}$$

2.4.3 YOLO

You only look once (YOLO) is a specialised CNN which is designed to improve object detection and classification.[Redmon et al., 2016] It was designed to be an exceedingly fast algorithm for object detection. YOLOv3 is specifically relevant as it is used in the application and is the third iteration of the algorithm. YOLOv3 can be split into two main parts, the feature extractor and the detector, as seen in Figure 18. This enables an image to go through the first stage to extract all the important features embedded in it. This can be applied to different scales of the initial image. The results are then passed onto the detector which uses the features in order to determine bounding boxes and classes.

The feature extractor uses Darknet-53, this is a CNN with a specific layout and has 53 layers. It also uses between 16 and 1024 filters at each of the 52 Convolutional layers. This enables the

Figure 18: YOLOv3 Architecture

algorithm to extract a range of features.[Redmon and Farhadi, 2018] Instead of having on feature vector produced after all the layers, for object detection yolov3 produces feature vectors from 3 different places. As can be seen in Figure 19, these three feature vectors care produces towards the end of the network and are then feed into the detector. Though the feature vector sizes depend on the image that is input to the feature extractor.

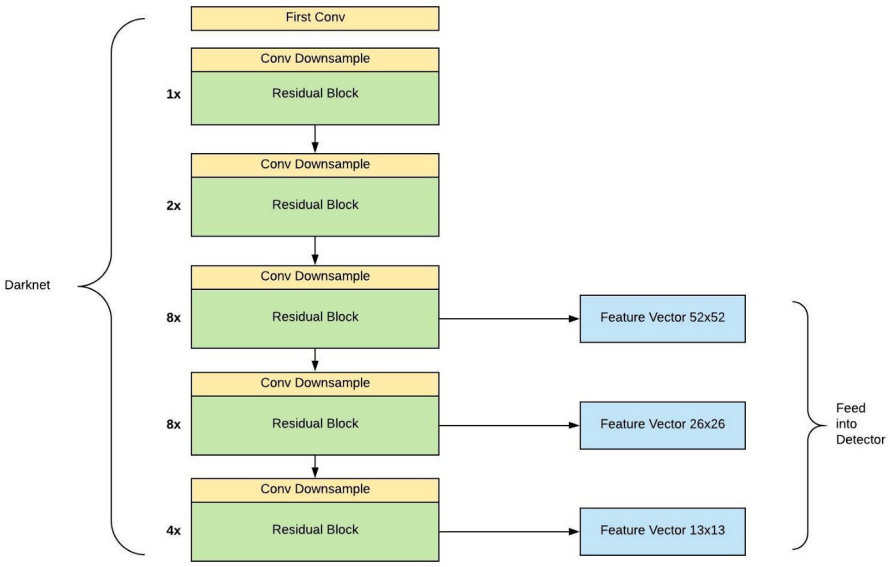


Figure 19: Darknet-53 Architecture

The detector is three sets of CNNs which can be combined to produce a series of results. The last vector to be extractor is put through a CNN with several convolutional layers before produces an output for large scale detections. In order for small or medium scale features to be detected the result of the first CNN is applied to the next feature vector and sent through another CNN. The process is repeated again in order to produce small-scale object detections.

2.5 Summary of Prior Work

As hinted at in the introduction and referenced in the rest of this report, there has been significant prior work on this project of which I have built on.[Denes, 2020, Pavlik, 2019] In order to tackle the application, [Denes, 2020] used YOLO to detect and classify seals. Whilst the detection of a seal against the background landscape was very successful, when it came to classifying Whitecoats, Moulted and Dead Pups, the results were not as good as hoped. There was a similar situation to the work by [Pavlik, 2019], the detection using ResNet was good, but the classification had a much lower accuracy. Both of these prior works used the algorithm to classify the seals from the initial stage, therefore creating a two-stage algorithm will hopefully have an improvement on the classification results.

3 Requirements Specification

The requirement for this project were developed throughout the year and in addition to the objectives that were specified in the Description, Objectives, Ethics and Resources (DOER) Report. The objectives that were specified are as follows.

3.1 Primary Objectives

- Research a variety of deep learning techniques on object detection and classification to see if the current strategy can be improved upon. Any research should be compared with the techniques used in the application.
- Prepare the dataset of provided seal images and background images to be several cropped images in order for them to be processed successfully.
- Implement a binary classifier that will detect a seal in an image in comparison to the background. Train a classifier which will use the detected seals from the binary classifier to recognise what type of seal (Whitecoat, Moulted, Dead Pup etc.) has been detected.

3.2 Secondary Objectives

- Calculate confidence values for the system as a whole which can be integrated into the population modelling.
- Experiment with the tiling approach of loading images in order to improve the performance of training.
- Experiment with the weighting of the classifier to favour the detection of pups.
- Integrate the results into the GIS system used by SMRU

The objectives specified at the beginning of the year are in priority order and also highlight the range of elements that needed to be worked on during the project. The requirements that follow go into more detail on the objectives that were initially specified.

3.3 Primary Requirements

- Research object detection Convolutional Neural Network (CNNs) based algorithms including YOLO and ResNet. This will be used in the first stage of the implementation
- Research machine learning algorithms ideal for classification such as Support Vector Machines and Random Forest. This will be used for classifying the images in the second stage of the implementation.
- For data preparation cropping of the larger TIFF files to small images will be required.
- Use binary classification for object detection of seals on images. A recall above 0.95 needs to be achieved before passing to the second stage.
- Transform the results of the first stage into cropped seal images and then use a Histogram of Oriented Gradients (HOG) to feed into the second stage.

- Use multiclass classification to detect the type of seal using the results of the first stage. A precision of over 0.80 should be achieved for each class.
- To evaluate the final results a precision-recall curve, ROC curve and confusion matrix should be produced for the results of the process overall.

3.4 Secondary Requirements

- Research alternate object detection methods including unsupervised options.
- Research and compare several transformation systems that could be added onto HOG for the training of the multiclass classifier.
- Use image augmentation to produce more training images for the second stage.
- For the multiclass classifier, compare the results against a CNN classifier which uses the cropped seal images instead.
- Evaluation of the system should include calculating confidence intervals for the precision and recall to see if it is consistent. Consistency could aid with calculation of populations.

3.5 Tertiary Requirements

- Connect the stages together so that training of both stages occurs as a whole system and subsequently can be applied as a whole to new aerial images.
- Integrate the resulting system into the GIS system used by SMRU.

4 Software Engineering Process

4.1 Development Process

Throughout the development of the project, I had weekly meetings with my supervisors in order to check in and keep an agile based approach. This enabled target setting each week for small, manageable goals to be completed. Weekly meetings were ideal as it provided an opportunity to raise concerns early and tweak the plan for the project if required. The agile process over the waterfall model was much preferred as it allowed small targets which would allow for balance of working on this project and any others in a given week. Also, as this project built upon pre-existing research it was ideal to complete additions step by step rather than use the waterfall model which is more suited when building a system from scratch. The agile method was also chosen as each stage could be developed separately and were not completely interlinked. The process also consisted of experimenting with a variety of possible algorithms and reporting results so the ability to change as development continued was vital.

4.2 Resources and Technologies

A variety of resources, tools and technologies were used throughout this project. The following specifies what was used as well as justification for each one.

Python: This was the language used throughout the project. There were two main reasons for this choice, the first was that all the existing code was in this language, so it was much easier to extend with the same language. The second reason was that python is ideal for machine learning projects due to the expansive libraries which can be utilised. For this project the main two libraries used were Scikit-Learn and NumPy. Scikit-Learn was used for experimentation and implementation of the second stage as it is easy to setup and run a variety of algorithms including SVM, Random Forest and simple neural networks. It also contains methods for evaluating the algorithm including confusion matrices. NumPy is a library which is used for any mathematical functions. This was used at all stages of the implementation from calculating the IoU to basic mathematical operations.

Jupyter Lab: This platform was used when developing the project. It allowed for a range of files to be used from an interactive jupyter notebook to standard python and text files. This was ideal for this project due to the different elements that needed to be pulled together at the different stages. The lab platform was intuitive to use and easy to pick up which was ideal when building on the prior work of this project.

TensorFlow: This is a library which was used for the first stage of the project. The reason for this was due to it being good at pipelining and highly parallel which, due to the large training dataset, increased efficiency and decreased training time.

External Server: The project was conducted using a server hosted by one of my supervisors, Kasim, which enabled the training to occur in a shorter duration than if run on a desktop machine. Due to the large dataset of images in the first stage, it would have taken weeks to train a model without the use of this server.

5 Ethics

There were no ethical considerations for this project as the aerial images were supplied by the SMRU and permission was given for them to be used.

6 Design

In this project it has been acknowledged several times that the final product has been produced as a result of building upon code which has previously tried to address the current problem. However, the strategy of this project is to address the problem in two stages rather than one algorithm which performs classification. The first stage is to use object detection to identify seals against the background. Aiming for a high recall on this first stage, the second stage is to take these identified images and classify them with a higher precision. A secondary aim is to see if there could be consistency in the precision and recall. If this is the case, then population estimates can be made with the project as it would be out by the same percentage each time. In regard to this project, the design was produced with remote connection in mind. Due to not being able to access the computer science labs in person, ssh tunnels would have to relied upon to connect to the external server.

6.1 Data Preparation

For any machine learning task data preparation is key as only with reliable training data can useful results be produced.

6.1.1 Image Cropping

The aerial images provided by the SMRU are far too large to enable an efficient training procedure. Therefore, the images need to be cropped into smaller tiles. Each of these tiles can then be fed through as part of training. Very little was altered in regard to this section of code but is an important part of the process so worth noting.

6.1.2 Tile Batches

When training the model, the tiles which are used are vital in determining how the model will perform. Files were created with a list of tiles to be used for training, valuation and testing. One avenue that was pursued as part of this project was to see if creating models for specific island(s) would improve the precision and recall of the final model. Therefore, lists were created for specific islands as well as an overall model which applied stratified sampling over all the models. The results of this are explained in the implementation.

6.1.3 Anchors

As a part of the YOLO parameters, anchors are required, which give the algorithm a sense of the size of bounding boxes that are expected to be predicted. For the majority of the time, it is recommended to use the default which is supplied alongside YOLO. However, I wanted to look into this as it seemed a potential route to improve the training of the algorithm. To do this, the anchors can be calculated by using k-means clustering method. This method will use Intersection over Union (IoU) as a metric in order to determine which cluster each point belongs to. The IoU is calculated for each of the boxes, in this case the bounding boxes around the seals, with each of the clusters. The process is repeated until every box remains in a cluster consistently. The clusters are then reported as the anchors to be used in training.

6.2 Object Detection

The first stage of the model is to detect seals in the tiles which make up the training set. The aim of the design is to provide as high a recall as possible so that the second stage will have all the possible

seal images and no seals are lost in detection. As has been seen in previous work [Pavlik, 2019, Denes, 2020], it is not too difficult to achieve this, but the precision will be compromised as a consequence. Therefore, if it is also possible to improve precision this will aid the second stage. The majority of the design in this section is to build on the code already produced and alter it to suit a two-stage process instead of the previously research single stage.

6.2.1 YOLO Architecture

As explained in the context survey, YOLO is CNN which detects objects by producing a bounding box around it and a probability that it is said object. In this project, YOLO is used to produce bounding boxes around potential seals. Lowering the threshold of the probability will enable a larger number of bounding boxes to be kept and therefore increase recall.

6.2.2 Training

Training datasets with recall in mind will focus on tweaking the YOLO parameters to produce this result. The parameters that will be focused on are:

- **Epochs:** The more epochs that are used the more opportunity the training process has to decrease the loss. Running the training for a higher number of epochs will increase the final precision but will consequently mean training takes longer.
- **NMS Threshold:** This is the IoU threshold, therefore this changes minimum overlap between bounding boxes in order for the predicted box to be considered correct.
- **Score Threshold:** This threshold is the minimum probability the bounding box can have and be kept as a prediction of a seal. It is important this threshold is low enough that as many seals can be detected as possible.
- **Optimizer:** A range of strategies are possible with YOLO including adam, momentum and sgd. It will be worth exploring each of these options to see which is most applicable to the scenario. In terms of the application, it seems as though momentum would be a good starting point.
- **Learning Rate Type:** The learning rate and type is vital as it determines how the model is changed in relation to the loss/errors in estimation.
- **Input Image Size:** The size of each tile impacts the training process and so evaluating different sizes to see which can capture the maximum seals is important. Larger tiles will prevent more seals being missed because they are cut between two tiles.
- **Batch Size:** The batches that the tiles are trained in will impact the calculation of loss and as a consequence the adjustment of parameters involved in training.

Due to the YOLO algorithm being implemented already; the design of this section is to focus of the parameters that influence recall and start with values which will benefit recall.

6.2.3 Evaluation

Alongside the current training script there is an accompanying evaluation script to evaluate the performance of the object detection. There is no need to adjust or change this script, but it will be used throughout the project to evaluate the object detection stage.

6.3 Transformation Stage

This stage of the process will take the bounding boxes of the predicted seals and transform this data such that it can be used to train the image classifier. Firstly, the predicted bounding boxes are used to cut a smaller, 80x80 tile, of the (predicted) seal. This image will then be transformed to a Histogram of Oriented Gradients (HOG) in order for it to be classified. This stage is used on training data in order to produce a training dataset for the classification stage but will also be used as part of a whole process to classify seals.

6.3.1 Prediction Cropping

Firstly, the predicted bounding boxes need to be transformed into a smaller tile. This method of cutting an image into a smaller tile is very similar to the initial image cropping. Therefore, it would be ideal to reuse this method for the smaller bounding boxes. The reason the tile should be around 80x80 is to prevent the seal not being included in the extracted tile or cut off at the edge. However, during implementation smaller tile sizes should be experimented with. This is because if a more cropped image can be produced then the HOG will be able to differentiate different seals more easily.

6.3.2 Extracting HOG

The second step of the transformation process is to extract the HOG from the smaller tiles and write this to a separate file. This can then be streamlined to be an input to the image classifier.

When preparing a training dataset for the image classifier the methodology required is slightly different. Firstly, several tiles of background and each type of seal will need to be passed through. For each one the HOG will be found but in addition the classification name of the tile will need to be added to a text file. This will then enable the image classifier to train based on the HOG values and the list of image classifications.

It will be worth experimenting with the extracting of data from each of the tiles to see which method is most effective. As well as HOG being extracted there is the option to look at RGB channel data or even greyscale pixel values. HOG has been effective in the past as a feature descriptor and given the low resolution of the images is a good place to start.
[Ebrahimzadeh and Jampour, 2014]

6.4 Image Classification

After transforming the smaller seal tiles, the aim of this stage is to classify the type of seal as well as any background tiles. Unlike with the first stage where the work was built on pre-existing code, this stage is entirely new and so requires thought during the design process. As discussed in the context survey above, there are many highly suitable options to explore. Due to the nature of this task I wished to take all of the mentioned options forward to the implementation stage to find the best suited method. This is due to all the options seeming equally valid and equally able to produce good results. When performing experiments to look at the hyperparameters of each technique and comparing each script, it is vital to use cross-validation to ensure the results/decisions are valid.

6.4.1 Support Vector Machine

The first method that was developed was to use a Support Vector Machine (SVM) which has previously been used alongside HOG to produce successful results.[Rybski et al., 2010] The SVM will take and split the training data from the transformation stage to train the image classifier. SVM

is highly supported through the Scikit-Learn library so it is fairly simple to implement. Initially it would be worth using this as a second stage object detector to ensure it works well and then alter as an image classifier. Adjusting the hyperparameters will impact the outcome of the script, it is worth exploring the following parameters:

- **kernel:** One of the key decisions involved with SVM is to decide the type of kernel to use. For this application it will probably be best to try rbf first and then compare against the other options. This is because rbf is the default set and would allow for the most flexible allocation of classification groups.
- **class_weight:** This is to adjust the balance of the training set. If the dataset from the transformation is unbalanced, then this hyperparameter is vital in ensure the results are not skewed by the input. An unbalanced training dataset is expected as the first stage has a high recall so a large proportion of the predicted bounding boxes will be background.
- **C:** This is the parameter which adjusts the precision of the model. This parameter determines how strongly (or not) misclassifications are considered in adjusting the model. The greater the value of this parameter, the less misclassifications should appear.
- **gamma:** This parameter is only relevant for the rbf kernel; however, it is of great importance as it is the inverse of the standard deviation of the rbf kernel function. A small gamma value will give a greater standard deviation and so greater variance. A larger gamma value will give a smaller variance. In application this parameter determines how much an individual training point impact the model as a whole. A small value means the impact is far reaching but a larger value means it only impact closer values.

6.4.2 Random Forest

The second method to develop was to use a Random Forest based script in order to produce an image classifier. Random Forest is a decision-based algorithm which uses the training dataset to produce many decision trees, these trees are then used to classify the image.[Navlani, 2018] This is a very different method to SVM and so a good alternative to try. Another reason for trying this method is that it is quite robust due to the number of decision trees used and does not suffer from overfitting due to taking the average of all the different predictions.[Navlani, 2018] Again, Random Forest is supported by the Scikit-Learn library, so it is relatively simple to transform to the required application. The key parameters to consider are explained below:

- **n_estimators:** The number of trees used to classify the images. The more trees that are used the longer the training process will take. Increases the number of trees should increase the accuracy of the algorithm but this will reach a maximum. Finding this maximum will involve several trials.
- **max_features:** This is how many features should be considered in the training process. As the HOG produces several features, reducing this could be beneficial. Therefore, exploring this option could adjust the accuracy.
- **class_weight:** Due to the training set being based on the results of the first stage, it may be required to adjust the weighting to have a more balanced training set. This is because with a high recall there could be far more background tiles than anything else. There are also a limited number of pup examples so adjusting to improve the detection of these could be vital.

- **bootstrap:** This is adjusting the method of sampling the training dataset. This specifies if a portion of the dataset is used or the whole thing.
- **max_depth:** This is another adjustment for the formation of the decision trees. The max_depth specifies how many layers (the depth) each decision tree can have. This could prevent unnecessarily large decision trees from being formed.

6.4.3 Neural Networks

Another option to consider was using a multi-layer perceptron(MLP) classifier. When conducting research around implementing this method, I also discovered using a Keras based NN and ResNet, a CNN. All three are slightly different and so warranted implementation to compare against the SVM and Random Forest scripts. Firstly, the multi-layer perceptron classifier trains using the backpropagation technique and uses Cross-Entropy as the loss function. The hyperparameters of interest are:

- **hidden_layer_sizes:** This determines the number of neurons in each of the hidden layers. Through this the number of layers is defined as well as the number of neurons. Tweaking these values will produce different results and so worth experimenting with.
- **activation:** This determines which of the activation functions to use for training the model. The activation function determines how the input is transformed into the output for each node in the network. An important choice which can completely change the neural network.
- **solver:** This determines which of the three solvers should be used to specify the weight optimisation for all nodes in order to reduce loss. As this will impact the loss and therefore the accuracy of the model, it is important that experimentation is done to choose the right solver.
- **learning_rate:** The learning rate determines at what frequency the loss is checked and the weights updated. Too small a value could mean the model takes a long time to train and may get stuck without the loss decreasing. Too large a value could result in non-ideal weights being determined as the training process takes a short amount of time.

Secondly, there is the option to use Keras, which is a library that uses TensorFlow in order to train a deep neural network. Through this you can specify the number of hidden layers and which activation function to use at each layer. Once this has been chosen then the next step is to compile the model by specifying the loss function and optimizer. The consideration of these hyperparameters is the same as those explained for the MLP Classifier.

Through research into Keras and previous work completed on this project[Pavlik, 2019], I discovered that Keras had pre-trained models for ResNet and ResNetV2. This would make it easy to adapt the scripts to the application that I needed. ResNet is also a type of CNN meaning that it differs in technique from the two previous neural network-based processes. Therefore, this could produce different results as it can produce a significantly deeper network. When using ResNet the aim would be to adapt the single stage ResNet script as previously developed[Pavlik, 2019]. When adapting there are two methods to consider, first simply using the output of the transformation stage to classify or to use the small tiles themselves as input to the image classifier. The reason to consider both options is that ResNet has been used as a successful image classifier[Han et al., 2019] which could be replicated in this project.

7 Implementation

This project is implemented in Python and makes use of the TensorFlow framework where applicable. The project is made up of a variety of scripts, some were created before this project and been tweaked based on the design and some have been created from scratch.

7.1 Data Preparation

7.1.1 Image Cropping

The image cropping scripts is called `ImageCrop` and was simply adjusted in order to cut the tiles required for training. The script had been created before this project.[Pavlik, 2019]

7.1.2 Tile Batches

Three main batches were implemented to look at how tiles from different islands impacted the result of the process as a whole. The first batch contains training, validation and test datasets from `FirFor` (Firth of Forth) tiles. This is a small set which would allow for training to occur quickly when trialling different hyperparameters, especially on the first stage. The second batch is a training, validation and test datasets from `Ork` (Orkney) which have similar tile backgrounds but a large number of images. This could confirm the results of the Firth of Forth batch and give a larger dataset to improve training at each stage. Finally, the third batch is a pre-existing set of files which is stratified sampling from all the different islands, including Orkney and Firth of Forth.[Denes, 2020]

7.1.3 Anchors

Implementation of this section was based on code previously created.[Denes, 2020][Nieradzik, 2018] This did have to be altered in order to work on the tile batches that had been created. The way the calculation of the anchors works is to use the size of the bounding boxes in the training dataset to calculate what the size of future bounding boxes looks like. The main error which occurred was that NaNs were often produced due to two of the chosen bounding boxes had the same size. Therefore, the code had to be tweaked to ensure the method would work correctly.

Experimentation with the specific anchors vs the pre-trained anchors supplied was conducted on the Firth of Forth batch. These anchors are:

Name	Anchors
yolo_anchors.txt	[[10,13], [16, 30], [33, 23], [30, 61], [62, 45], [59, 199], [156, 198], [373, 326]]
firth_anchors.txt	[[32,60], [35, 60], [60, 35], [43, 60], [60, 45], [48, 60], [55, 60], [60, 60]]

The results of the experiment are as follows:

Anchor	Score Threshold	Precision	Recall
yolo_anchors.txt	0.2	0.3368	0.8476
yolo_anchors.txt	0.005	0.0047	0.9492
firth_anchors.txt	0.2	0.1887	0.5139
firth_anchors.txt	0.005	0.005	0.9710

The results of this experiment are contrasting as for a 0.2 score threshold the yolo anchors has a greater precision and recall but the firth anchors for a 0.005 score threshold has a greater recall. The latter result could perhaps be because of the small training dataset. Therefore, despite the specialised anchors it would be better to use the supplied yolo anchors. The reason this is the case

is because the bounding boxes are very similar, so the anchors produced are very similar which does not improve the detection of seals.

7.2 Object Detection

Similar to the Data Preparation section, the work on object detection was building onto the pre-existing code to ensure it has a high recall. If the precision can be relatively good, then this would also assist the second stage, but the priority is definitely with the recall.

7.2.1 Training

The key part of training algorithms to produce a high recall and reasonable precision is to alter the hyperparameters within the `args.py` script. The main hyperparameter to change was `score_threshold` as this adjusts the minimum probability the bounding boxes have to be to a positive detection. Decreasing this parameter to 0.01 produces a recall around 0.96-0.99 for the variety of models tested. For the final model the IoU threshold was set to 0.45, the momentum optimizer was selected with a piecewise constant strategy. The image sizes of each tile were 416x416 and the maximum batch size that could be run was 32. One useful feature is the ability to evaluate the model against the training set after a specified number of epochs. This enables checking the aP and maP to ensure the model is not overfitted. Training a model using these settings took 2-3 days.

7.2.2 Evaluation

The evaluation conducted in the `eval.py` script was pre-existing and simply required altering the parameters in order to evaluate the model produced. The only major changes to this script occurred later when implementing the transformation stage. The results are shown in the table below.

Threshold	Precision	Recall	aP
0.01	0.0143	0.9960	0.9141
0.1	0.2836	0.9681	0.9100
0.2	0.5760	0.9402	0.8972
0.3	0.0.7627	0.8964	0.8672
0.4	0.8629	0.8466	0.8267
0.5	0.9164	0.7859	0.7723
0.6	0.9593	0.7042	0.6956
0.7	0.9752	0.6275	0.6214
0.8	0.9835	0.4761	0.4729
0.9	0.9957	0.2281	02273

To keep as high a recall as possible the 0.01 threshold is used when identifying tiles for the second stage. However, it may be worth trying 0.1 or 0.2 as a threshold in the future.

7.3 Transformation Stage

7.3.1 Prediction Cropping

Cropping is linked to the `eval.py` script as the chosen model produces predicted bounding boxes which can be transformed for the second stage. Therefore, the code to execute this is in the `eval_utils.py` script and the `pos_seal_extraction` method. This method takes the predicted

bounding boxes, actual bounding boxes and location to put the images. The predicted bounding boxes are compared with the actual ones using the IoU metric, this determines if the image is a seal or not. Based on the supplied training or validation file the seal is classified and put into the correct folder (Moulted, Whitecoat, Dead Pup etc.). Each bounding box will be of a particular size, but the tiles should all be the same size. Therefore a buffer in the x and y directions are added. If the tile needs to be 80x80 pixels for example, then the number of pixels above the current bounding box size are split in half and added to each side of the bounding box. This keeps the original bounding box in the centre so there is more of a chance of the entire seal being in the tile. Tile sizes were tested from a 60x60 tile up to an 80x80 tile to see what impact this had on the second stage. The following were the results when used with HOG and SVM:

Tile Size	Precision	Recall	Accuracy
60x60	0.47	0.37	0.37
70x70	0.46	0.37	0.37
80x80	0.45	0.38	0.38

As can be seen, there is not much difference in the results due to the tile size. Adding more pixels does not hinder the picking out of the seal and restricting to the bounding box size doesn't cut off part of the seal. Therefore, the 60x60 size was used going forwards.

7.3.2 Extracting HOG

The script in which this part of the transformation stage takes place is `extract.py` and has been adapted from a script provided by one of my supervisors, Kasim Terzíc. This script initially produced data from the HOG and RGB channels for each tile. This was adapted to allow experimentation of just HOG values. The script uses the path of the images once they have been cropped to extract the features and then save this in an appropriate place. This can then be accessed by the training script for the classification stage. In order to check which type of data was the most effective to be extracted from the features, a series of tests were completed once the implementation of the image classifier was complete. Comparing using RGB on the SVM implementation it was clear that the colour channels impacted the classification of the whitecoats (due to their white coat standing out from the green or light brown background). However, without using RGB this does not clearly show and so has a significant effect on the results.

7.4 Image Classification

This section looks at the implementation of the different second stage algorithms. The work on this section was all new code. The following provides an overview of what was added and some of the results.

7.4.1 Support Vector Machine

The implementation of this algorithm can be found in the `train_svm.py` script. This uses specified input files (the results of `extracted_features.py`) in order to train the SVM. In order to find the best hyperparameters for this application, the scikit-optimize library was used with Bayesian Optimization. This enables a variety of different hyperparameters to be tested and the best ones be reported as a result. In a similar manner to the first stage, the more input images are supplied the longer the algorithm takes to train. For the case of the SVM algorithm, it typically took 5 minutes to train as an image classifier. The issue came when tuning the hyperparameters as it could lead to

100s of fits due to the large range of options. The following is the results of the tuning.

```
C = 1, degree = 1, gamma = 0.0001, kernel = rbf
```

Class	Precision	Recall	f1-score
Whitecoats	0.53	0.32	0.40
Moulted	0.25	0.31	0.28
Dead Pups	0.02	0.46	0.30

If there was more time it would have been good to tune the hyperparameters further. However, the current hyperparameters lead to a great recall of the dead pups which has often been a challenge.[Denes, 2020, Pavlik, 2019]

7.4.2 Random Forest

The implementation of the random forest classifier can be found in the `train_rf.py`. In a similar manner to the SVM implementation, the random forest implementation uses `GridSearchCV` in order to tun the hyper parameters. The hyperparameter that were trained the best were `max_depth=90, max_features='sqrt', min_samples_leaf=6, min_samples_split=10, n_estimators=1000`. This resulted in the following results:

Class	Precision	Recall	f1-score
Whitecoats	0.53	0.49	0.51
Moulted	0.00	0.00	0.00
Dead Pups	0.00	0.00	0.00

As can be seen the hyperparameter tuning led to the focus being on the whitecoats and as a result nothing is ever classed as a dead pup or moulted seal.

7.4.3 Neural Networks

The initial implementation of a NN is using a multi-layer perceptron. This manual method enables the choice of different layers and which activation function to use. Given the HOG data I decided to train a two hidden layer network with an 'adam' solver. The results are as follows:

Class	Precision	Recall	f1-score
Whitecoats	0.46	0.63	0.53
Moulted	0.26	0.10	0.14
Dead Pups	0.06	0.02	0.3

Similar to the SVM and RF algorithms, the hyperparameters took some tuning in order to find the best performance. However, the performance was not as good as the SVM implementation

Unfortunately, there was not time for training with a Keras based algorithm or a ResNet based algorithm. If there had been then it would have been good to add these as alternatives.

8 Evaluation and Critical Appraisal

Overall, this project asked for an understanding of machine learning, deep learning as well as how to apply it to research code. Working with previous code was more of a challenge than I had initially anticipated. With more time I would have worked on the second stage further and explored additional options including ResNet, and further hyperparameter work. However, this project has looked into different options for a two-stage implementation. The main surprise when exploring this implementation has been how unsuccessful hyperparameter tuning has been. When completing vast searches, the results end with only the whitecoats class giving good results and the other classes being overlooked. The training time for the first stage and then the hyperparameter tuning has taken so long that unfortunately not all the primary requirements of the project have been met. With more time I am sure that more of the requirements could have been met and the results of the image classification improved.

As briefly highlighted, future work on this project is possible by focusing in on the second stage of the process and working on the different algorithms to improve performance. This can be achieved in several ways. Firstly, research into an alternative to the HOG method could be done or an image-based algorithm such as ResNet or a R-CNN could be used directly on the smaller tiles. Secondly work on the image classification algorithms to fine tune it to perform better. Finally, this problem is a difficult one due to the seals being the same colour and shape as the rocks behind them. Whilst there can be improvement made upon the application of this project, it would be difficult to predict by how much.

References

[Ahmadzadeh et al., 2017] Ahmadzadeh, A., Kempton, D. J., Schuh, M. A., and Angryk, R. A. (2017). Improving the functionality of tamura directionality on solar images. In *2017 IEEE International Conference on Big Data (Big Data)*, pages 2518–2526. IEEE.

[Albawi et al., 2017] Albawi, S., Mohammed, T. A., and Al-Zawi, S. (2017). Understanding of a convolutional neural network. In *2017 International Conference on Engineering and Technology (ICET)*, pages 1–6. Ieee.

[Caie et al., 2021] Caie, P. D., Dimitriou, N., and Arandjelović, O. (2021). Precision medicine in digital pathology via image analysis and machine learning. In *Artificial Intelligence and Deep Learning in Pathology*, pages 149–173. Elsevier.

[Commons, 2018] Commons, W. (2018). Neuron and myelinated axon, with signal flow from inputs at dendrites to outputs at axon terminals. File:Neuron3.png.

[Denes, 2020] Denes, D. (2020). *Neural Networks and Deep Learning*. PhD thesis, University of St. Andrews.

[Ebrahimzadeh and Jampour, 2014] Ebrahimzadeh, R. and Jampour, M. (2014). Efficient handwritten digit recognition based on histogram of oriented gradients and svm. *International Journal of Computer Applications*, 104(9).

[Eliassy, 2019] Eliassy, E. (2019). Create your own image classifier with angular and tensorflow.js.

[Gandhi, 2018] Gandhi, R. (2018).

[Girshick et al., 2014] Girshick, R., Donahue, J., Darrell, T., and Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 580–587.

[Golgi, 1873] Golgi, C. (1873). Sulla sostanza grigia del cervello. *Gazetta Medica Italiana*, 33:244–246.

[Han et al., 2019] Han, L., Yu, C., Xiao, K., and Zhao, X. (2019). A new method of mixed gas identification based on a convolutional neural network for time series classification. *Sensors*, 19:1960.

[Hyndman and Athanasopoulos, 2018] Hyndman, R. J. and Athanasopoulos, G. (2018). Forecasting: Principles and practice (2nd ed).

[Kotu and Deshpande, 2019] Kotu, V. and Deshpande, B. (2019). Chapter 10 - deep learning. In Kotu, V. and Deshpande, B., editors, *Data Science (Second Edition)*, pages 307–342. Morgan Kaufmann, second edition edition.

[Learning*, 2018] Learning*, A. i. M. (2018). The vanishing gradient problem and relus – a tensorflow investigation.

[Li and Zhou, 2019] Li, X. and Zhou, Z. (2019). Object re-identification based on deep learning. In *Visual Object Tracking with Deep Neural Networks*. intechopen.

[Liu, 2018] Liu, Y. (2018). The confusing metrics of ap and map for object detection / instance segmentation.

[McCormick, 2013] McCormick, C. (2013). Gradient vectors.

[McCulloch and Pitts, 1943] McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133.

[Mr.A and lnathan, 2017] Mr.A and lnathan (2017). What is a threshold in a precision-recall curve?

[Navlani, 2018] Navlani, A. (2018). Random forests classifiers in python.

[Nielsen, 2015] Nielsen, M. A. (2015). *Neural networks and deep learning*, volume 25. Determination press San Francisco, CA.

[Nieradzik, 2018] Nieradzik, L. (2018). kmeans-anchor-boxes. https://github.com/lars76/kmeans-anchor-boxes.

[Noriega, 2005] Noriega, L. (2005). Multilayer perceptron tutorial. *School of Computing. Staffordshire University.*

[Pavlik, 2019] Pavlik, S. (2019). *Deep learning for automatic seal counting in aerial images.* PhD thesis, University of St. Andrews.

[Point, 2021] Point, J. (2021). Single layer perceptron in tensorflow - javatpoint.

[Redmon et al., 2016] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788.

[Redmon and Farhadi, 2018] Redmon, J. and Farhadi, A. (2018). Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767.*

[Rosenblatt, 1958] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386.

[Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *nature*, 323(6088):533–536.

[Russell and Norvig, 2002] Russell, S. and Norvig, P. (2002). *Artificial intelligence: a modern approach.* Pearson.

[Rybski et al., 2010] Rybski, P. E., Huber, D., Morris, D. D., and Hoffman, R. (2010). Visual classification of coarse vehicle orientation using histogram of oriented gradients features. In *2010 IEEE Intelligent vehicles symposium*, pages 921–928. IEEE.

[Sauer, 2020] Sauer, A. (2020). Social network for programmers and developers.

[Scot, 2020] Scot, N. (2020). Seals.

[University, 2021] University, S. (2021).

[Walters, 2019] Walters, A. (2019). image-filter-matrix-cnn.

[Weng, 2017] Weng, L. (2017). Object detection for dummies part 1: Gradient vector, hog, and ss.

[y Cajal, 1888] y Cajal, S. R. (1888). *Estructura de los centros nerviosos de las aves.*